

OPERATING SYSTEM

Chapter 6 — Synchronization Tools & Semaphore

University Exam Notes

Institute of Information Technology, Jahangirnagar University
Based on slides by Md. Mahmudur Rahman (Assistant Professor, IIT)

1. Semaphore

1.1 Definition

A semaphore is a shared integer variable used to manage access to common resources in a concurrent system.

Proposed by: Dijkstra in 1965.

Purpose: Solve critical section problems and achieve process synchronization.

1.2 Main Idea

- Semaphore is just an integer variable shared between threads/processes.
- It controls access to a resource.
- Two operations are used: wait() (also called P) and signal() (also called V).
- Both operations are atomic — no interruption in between.

1.3 Types of Semaphore

Feature	Binary Semaphore	Counting Semaphore
Value Range	0 or 1 only	0 to N (any positive integer)
Also Known As	Mutex Lock	General Semaphore
Initial Value	1	Number of available resources (N)
Use Case	Mutual exclusion (1 process at a time)	Multiple instances of a resource
Example	One printer used by many processes	4 database connections
Overflow/Underflow?	Yes, possible	Yes, possible

1.4 P and V Operations (wait & signal)

wait() — also called P operation or down operation

```
wait(S)
{
    while(S <= 0);    // busy waiting: loop until resource is available
    S--;              // decrease count: one resource taken
}
```

- **while(S<=0)** → Process waits here if no resource is free.
- **S--** → Resource count decreases by 1 (process takes a resource).

signal() — also called V operation or up operation

```
signal(S)
{
    S++;    // increase count: resource released back
}
```

- **S++** → Resource count increases by 1 (process releases a resource).

1.5 Atomic Operation

Atomic means: read → modify → update happens as ONE step. No other process can interrupt in between.

- Both wait() and signal() are atomic operations.
- This prevents race conditions.
- No preemption during these operations.

1.6 Critical Section

- Critical Section = part of code where shared resources are accessed.
- Only ONE process can be in critical section at a time.
- Surrounded by wait() on entry and signal() on exit.

Flow: Process enters → wait(S) → [Critical Section] → signal(S) → exits

*[Insert Image Here: Critical Section with P and V Operations]
(Page Suggestion: Slide 3)*

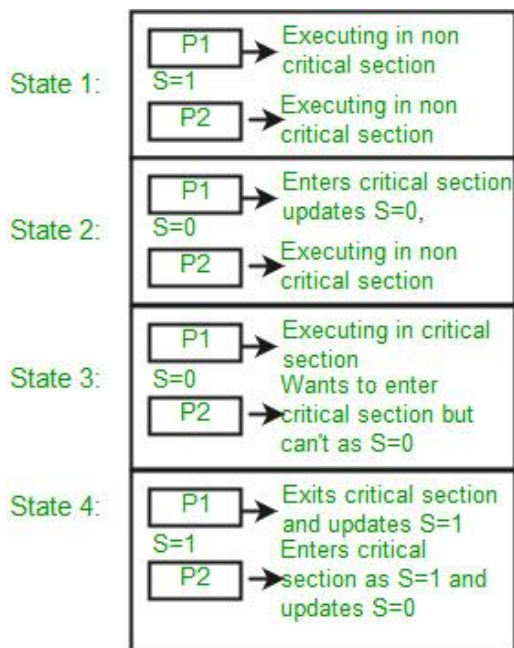
1.7 Mutual Exclusion

- Mutual Exclusion = only ONE process enters the critical section at a time.
- Semaphore ensures mutual exclusion.

Example: Two Processes P1 and P2 (s = 1 initially)

Step	P1 Action	Semaphore (s)	P2 Action
1	Calls wait(s)	s becomes 0	Wants to enter CS
2	Enters Critical Section	s = 0	Blocked (s <= 0, waits)
3	Exits CS, calls signal(s)	s becomes 1	Can now proceed

Step	P1 Action	Semaphore (s)	P2 Action
4	—	s = 0 again	Enters CS



1.8 Busy Waiting

- **Definition:** Process keeps looping (while loop) instead of sleeping while waiting for resource.
- Also called: Spin Lock.
- Problem: Wastes CPU time — process does no useful work while waiting.
- Solution: Use sleep/wakeup mechanism instead (blocking semaphore).

Feature	Busy Waiting	Blocking (Sleep)
CPU Usage	High (wastes cycles)	Low (process sleeps)
Complexity	Simple	More complex
Used In	Short waits	Long waits

1.9 Important Exam Points — Semaphore

- Semaphore proposed by Dijkstra in 1965.
- Two types: Binary (0 or 1) and Counting (0 to N).
- P = wait = down; V = signal = up.
- Both P and V are atomic.
- Semaphore initialized to 1 for binary, N for counting.
- Busy waiting = spinning in while loop.

1.10 Common Mistakes

- Confusing P and V: P decrements (waits), V increments (signals).
- Forgetting initialization: Binary starts at 1, not 0.
- Not knowing busy waiting = wasted CPU time.

1.11 Quick Revision — Semaphore

Semaphore = shared integer | Binary (0/1) | Counting (0 to N) | wait()=P=decrement | signal()=V=increment | Both atomic | Controls critical section access.

2. Binary Semaphore

2.1 Definition

A Binary Semaphore is a semaphore that can only take values 0 or 1. It acts like a mutex lock.

2.2 Main Idea

- Value = 1 → resource is FREE.
- Value = 0 → resource is BUSY (someone is using it).
- Initialized to 1 at the start.
- Used for mutual exclusion: only 1 process in critical section.

2.3 Working Process

1. Semaphore $s = 1$ (initialized).
2. Process P1 calls wait(s): s becomes 0 → P1 enters critical section.
3. Process P2 calls wait(s): $s = 0$, so P2 waits (busy waiting).
4. P1 finishes and calls signal(s): s becomes 1.
5. P2 can now proceed: s becomes 0 again → P2 enters critical section.

Flow: $s=1 \rightarrow P1: wait() \rightarrow s=0 \rightarrow CS \rightarrow signal() \rightarrow s=1 \rightarrow P2$ enters

2.4 Code

```
Semaphore s = 1;

// Process P1
wait(s);           // s becomes 0
// critical section
signal(s);          // s becomes 1

// Process P2
wait(s);           // s was 1, now 0 (or waits if 0)
// critical section
signal(s);          // s becomes 1
```

2.5 Advantages & Problems

Advantages	Problems
Simple to implement	Busy waiting wastes CPU
Ensures mutual exclusion	Only 1 process at a time (not suitable for multi-resource)

Advantages	Problems
Prevents race conditions	Deadlock possible if not used correctly

2.6 Quick Revision — Binary Semaphore

Binary Semaphore: values 0 or 1 | Init=1 | 1=free, 0=busy | wait() sets 0, signal() sets 1 | Also called Mutex Lock

3. Counting Semaphore

3.1 Definition

A Counting Semaphore is a semaphore with a value ranging from 0 to N. It controls access to multiple instances of a resource.

3.2 Main Idea

- Initialized to N = number of available resource instances.
- Each wait() decrements the count (one resource taken).
- Each signal() increments the count (one resource released).
- When count = 0, all resources are busy — processes must wait.

3.3 Working Process

Example: 4 resource instances (S = 4), 5 processes P1–P5

Event	Action	S value
Start	S initialized	4
P1 calls wait()	P1 gets resource	3
P2 calls wait()	P2 gets resource	2
P3 calls wait()	P3 gets resource	1
P4 calls wait()	P4 gets resource	0
P5 calls wait()	P5 WAITS — no resource	0 (blocked)
P1 calls signal()	P1 releases resource	1
P5 unblocked	P5 gets resource	0

3.4 Overflow and Underflow

Condition	Overflow	Underflow
Occurs When	Producer adds to full buffer	Consumer reads from empty buffer
Result	Data overwritten / lost	Wrong data / crash

Condition	Overflow	Underflow
Semaphore Check	wait(E) — check empty slots	wait(I) — check filled items
Fix	Block producer when full	Block consumer when empty

3.5 Quick Revision — Counting Semaphore

Counting Semaphore: values 0 to N | Init=N (resource count) | wait()=decrement | signal()=increment | All N=0 → wait | Used for multiple resource instances

4. Producer-Consumer Problem

4.1 Definition

Producer-Consumer is a classic synchronization problem where one process (producer) writes data and another (consumer) reads data from a shared buffer.

4.2 Main Idea

Role	What it does	Problem if unsynchronized
Producer	Creates items, puts into buffer	May overflow the buffer
Consumer	Takes items from buffer, uses them	May underflow (read empty buffer)
Buffer	Shared storage (size N)	Race condition on buffer access

4.3 Problems to Solve

Producer Must Handle:

- **Overflow Check:** Is the buffer full? If yes, wait.
- **Mutual Exclusion:** No consumer should access buffer at same time.
- **Buffer Update:** Add item and update count.

Consumer Must Handle:

- **Underflow Check:** Is the buffer empty? If yes, wait.
- **Mutual Exclusion:** No producer should access buffer at same time.

- **Buffer Update:** Remove item and update count.

4.4 Without Semaphore — The Race Condition

- Producer increments count; consumer decrements count.
- If both do this simultaneously — count becomes wrong (race condition).
- Shared buffer = critical section.
- Solution: Use semaphores for synchronization.

4.5 Producer-Consumer Using Semaphore

Variables:

Variable	Meaning	Initial Value
E (Empty)	Number of empty slots in buffer	N (buffer size)
S (Semaphore)	Mutex — controls CS access	1
I (Items)	Number of filled slots in buffer	0

Producer Code:

```
do {
    wait(E);      // overflow check: wait if buffer is full (E=0)
    wait(S);      // entry section: lock the critical section (mutex)
    add();        // CRITICAL SECTION: add item to buffer
    signal(S);    // exit CS: unlock mutex
    signal(I);    // increase filled item count (tell consumer)
} while (true);
```

Line-by-Line Explanation — Producer:

- **wait(E):** E = empty slots. If E=0, buffer is full → producer WAITS. Decrements E.
- **wait(S):** S = mutex. Lock the critical section so no one else can enter.
- **add():** Add item to buffer (critical section work).
- **signal(S):** Unlock the critical section — others can now enter.
- **signal(I):** Increment I = one more item is available for consumer.

Consumer Code:

```
do {
    wait(I);      // underflow check: wait if buffer is empty (I=0)
    wait(S);      // entry section: lock the critical section (mutex)
    consume();    // CRITICAL SECTION: remove item from buffer
    signal(S);    // exit CS: unlock mutex
    signal(E);    // increase empty slot count (tell producer)
} while (true);
```

Line-by-Line Explanation — Consumer:

- **wait(I):** I = filled items. If I=0, buffer empty → consumer WAITS. Decrements I.
- **wait(S):** Lock the critical section.
- **consume():** Remove item from buffer (critical section work).

- **signal(S)**: Unlock the critical section.
- **signal(E)**: Increment E = one more empty slot available for producer.

4.6 Flow Diagram

Producer: $\text{wait}(E) \rightarrow \text{wait}(S) \rightarrow \text{add}() \rightarrow \text{signal}(S) \rightarrow \text{signal}(I)$

Consumer: $\text{wait}(I) \rightarrow \text{wait}(S) \rightarrow \text{consume}() \rightarrow \text{signal}(S) \rightarrow \text{signal}(E)$

4.7 How Semaphore Solves the Problem

- S (mutex) ensures only ONE process (producer or consumer) is in critical section at a time.
- E prevents overflow: producer waits when buffer is full.
- I prevents underflow: consumer waits when buffer is empty.
- After exiting CS, semaphore is updated to allow others to proceed.

4.8 Important Exam Points — Producer-Consumer

- Three semaphores: E (empty), S (mutex), I (items).
- Producer: wait(E) first, Consumer: wait(I) first.
- Both call wait(S) to enter critical section.
- Overflow = buffer full | Underflow = buffer empty.
- signal() is called AFTER CS to release the mutex.

4.9 Quick Revision — Producer-Consumer

$E = \text{empty slots}(\text{init} = N) \mid S = \text{mutex}(\text{init} = 1) \mid I = \text{items}(\text{init} = 0) \mid$ Producer: $\text{wait}(E) \rightarrow \text{wait}(S) \rightarrow \text{add} \rightarrow \text{signal}(S) \rightarrow \text{signal}(I) \mid$ Consumer: $\text{wait}(I) \rightarrow \text{wait}(S) \rightarrow \text{consume} \rightarrow \text{signal}(S) \rightarrow \text{signal}(E)$

5. Reader-Writer Problem

5.1 Definition

The Reader-Writer problem deals with shared data where multiple readers can read simultaneously, but only one writer can write at a time, and readers and writers cannot work together.

5.2 Main Idea — Analogy

Analogy: A notice board. Teacher (Writer) puts a notice — students (Readers) cannot read while teacher writes. But multiple students can read at the same time.

5.3 Rules

Scenario	Reader Action	Writer Action
Writer in CS	Cannot enter CS	One writer only in CS
Reader in CS	Multiple readers allowed in CS	Cannot enter CS
No one in CS	Can enter freely	Can enter freely

5.4 Variables

Variable	Meaning	Initial Value
write	Semaphore for writer access	1
read	Semaphore for protecting read_count	1
read_count	Number of readers currently reading	0

5.5 Reader-Writer Code Using Semaphore

Writer Code:

```
wait(write);          // entry: lock so no reader/writer enters
writeFunction();      // CRITICAL SECTION: write to shared resource
signal(write);        // exit: release lock — others can now enter
```

Line-by-Line Explanation — Writer:

- **wait(write):** Lock the resource — no reader or other writer can enter.
- **writeFunction():** Do the actual write operation.
- **signal(write):** Release the lock — allow readers or writers to proceed.

Reader Code:

```
wait(read);           // lock read_count (mutual exclusion on counter)
read_count++;         // one more reader arrived
if (read_count == 1) { // first reader? then block the writer
    wait(write);       // no writer can enter now
}
signal(read);         // release read_count lock — other readers can update
count

readFunction();        // CRITICAL SECTION: read the shared resource

wait(read);           // lock read_count for exit update
read_count--;         // one reader leaving
if (read_count == 0) { // last reader leaving? let writer in
    signal(write);     // writer can now enter
}
signal(read);         // release read_count lock
```

Line-by-Line Explanation — Reader:

- **wait(read):** Lock the read_count variable (prevent race on counter).
- **read_count++:** Increment: one more reader is active.
- **if(read_count==1):** If this is the FIRST reader, block the writer with wait(write).
- **signal(read):** Release read_count lock — other readers can update.
- **readFunction():** Do the actual read (multiple readers can be here).
- **read_count--:** Decrement: this reader is done.
- **if(read_count==0):** If LAST reader leaving, signal(write) — writer can now enter.

Writer	Reader
<pre>wait (write); //Entry section writeFunction (); //CS signal (write); //Exit section</pre>	<pre>wait (read); //Entry section read_count++ if (read_count==1) { wait (write); //no writer can enter } signal (read); //multiple reader can access readFunction () // CS wait (read); //Exit section read_count-- if (read_count==0) { signal (write); //now writer can enter } signal (read);</pre>

5.6 Answer to Slide 12 Questions

Question	Answer
Reader in CS → another reader?	YES — multiple readers allowed simultaneously
Reader in CS → writer enters?	NO — wait(write) is held by first reader
Writer in CS → reader enters?	NO — write semaphore is locked
Writer in CS → another writer?	NO — write semaphore is locked

5.7 Important Exam Points — Reader-Writer

- Multiple readers can be in CS at the same time.
- Only 1 writer can be in CS at a time.
- read_count tracks number of active readers.
- First reader blocks writer (wait(write)). Last reader unblocks writer (signal(write)).
- read semaphore protects read_count — not the CS itself.

5.8 Quick Revision — Reader-Writer

write=1 | read=1 | read_count=0 | Writer: wait(write)→write→signal(write) | Reader: update read_count→if first, wait(write)→read→if last, signal(write)

6. Practice Problem — ATM Booth

6.1 Problem Statement

Multiple users can withdraw money at the same time using ATMs. BUT only ONE person can load money (refill) inside the ATM, and NO user can withdraw during loading.

6.2 Analysis

Role	Maps To	Rule
ATM User (withdraw)	Reader	Multiple users can withdraw simultaneously
Money Loader (refill)	Writer	Only 1 loader at a time; no users during loading

This is a Reader-Writer problem! Users = Readers. Money Loader = Writer.

6.3 Solution Variables

Variable	Meaning	Init
atm	Semaphore for loader access	1
read	Semaphore for protecting user_count	1
user_count	Number of users currently withdrawing	0

6.4 Solution Code

Money Loader (Writer):

```
wait(atm);          // entry: block all users while loading
loadMoney();        // CRITICAL SECTION: load money into ATM
signal(atm);        // exit: allow users to withdraw again
```

ATM User (Reader):

```
wait(read);          // lock user_count
user_count++;        // one more user withdrawing
if (user_count == 1) { // first user: block loader
    wait(atm);
}
signal(read);        // release user_count lock

withdraw();          // CRITICAL SECTION: withdraw money

wait(read);          // lock user_count for exit
user_count--;        // user done
if (user_count == 0) { // last user: allow loader
    signal(atm);
}
signal(read);        // release user_count lock
```

6.5 Key Insight

- This problem is structurally identical to the Reader-Writer problem.
- ATM Users = Readers (many allowed at once).

- Money Loader = Writer (exclusive access, blocks all users).
- First user blocks loader; last user unblocks loader.

6.6 Quick Revision — ATM Problem

ATM Problem = Reader-Writer Problem | Users=Readers | Loader=Writer | Multiple users can withdraw simultaneously | Loader needs exclusive access (no users allowed)

7. One-Page Quick Revision Sheet

All Key Concepts at a Glance

Topic	Key Point
Semaphore	Shared integer variable — controls access to resources
Proposed by	Dijkstra (1965)
Binary Semaphore	Values: 0 or 1. Init=1. Also called Mutex Lock.
Counting Semaphore	Values: 0 to N. Init=N (number of resources).
wait() / P	Decrement S. If $S \leq 0 \rightarrow$ wait. Atomic.
signal() / V	Increment S. Atomic.
Atomic	Read+Modify+Update in one step, no interruption.
Critical Section	Code that accesses shared resource. Only 1 at a time.
Mutual Exclusion	Prevent 2+ processes from entering CS simultaneously.
Busy Waiting	Process loops waiting — wastes CPU.
Overflow	Producer adds to full buffer $\rightarrow$ blocked by wait(E).
Underflow	Consumer reads empty buffer $\rightarrow$ blocked by wait(I).
P-C Semaphores	$E = \text{empty}(N)$, $S = \text{mutex}(1)$, $I = \text{items}(0)$.
Producer Flow	$\text{wait}(E) \rightarrow \text{wait}(S) \rightarrow \text{add}() \rightarrow \text{signal}(S) \rightarrow \text{signal}(I)$
Consumer Flow	$\text{wait}(I) \rightarrow \text{wait}(S) \rightarrow \text{consume}() \rightarrow \text{signal}(S) \rightarrow \text{signal}(E)$
Reader-Writer	Many readers OR one writer — never both.
Reader flow	$\text{wait}(\text{read}) \rightarrow \text{read_count}++ \rightarrow$ if 1st: $\text{wait}(\text{write}) \rightarrow \text{signal}(\text{read}) \rightarrow \text{read} \rightarrow \text{wait}(\text{read}) \rightarrow \text{read_count}-- \rightarrow$ if 0: $\text{signal}(\text{write}) \rightarrow \text{signal}(\text{read})$
Writer flow	$\text{wait}(\text{write}) \rightarrow \text{write} \rightarrow \text{signal}(\text{write})$
ATM Problem	Users=Readers, Loader=Writer. Same as Reader-Writer.

8. Important Definitions Table

Term	Simple Definition
Semaphore	A shared integer used to control access to common resources.
Binary Semaphore	Semaphore with only 0 or 1 value. Acts like a lock.
Counting Semaphore	Semaphore with value 0 to N. Controls multiple resource instances.
wait() / P	Operation that decrements semaphore; makes process wait if resource is unavailable.
signal() / V	Operation that increments semaphore; signals that resource is now available.
Atomic Operation	An operation that completes without any interruption (all or nothing).
Critical Section	Part of code where shared resources are accessed.
Mutual Exclusion	Guarantee that only ONE process is in the critical section at a time.
Busy Waiting	A process continuously checks (loops) if it can proceed — wastes CPU.
Race Condition	Two processes access shared data simultaneously → unpredictable result.
Overflow	Trying to add to a full buffer — results in data loss.
Underflow	Trying to consume from an empty buffer — results in error.
Process Synchronization	Coordinating multiple processes so they work correctly on shared data.
Mutex	Mutual Exclusion object — same as Binary Semaphore in concept.

9. Important Comparison Tables

9.1 Binary vs Counting Semaphore

Feature	Binary Semaphore	Counting Semaphore
Value Range	0 or 1 only	0 to N
Initial Value	1	N (number of resources)
Also Called	Mutex Lock	General Semaphore
Use Case	Mutual exclusion (1 process only)	Multiple resource instances
Example	Printer, one shared variable	Database connections, buffer slots
Overflow/Underflow Risk	Yes	Yes

9.2 wait() vs signal()

Feature	wait() / P	signal() / V
Action	Decrements S	Increments S
Effect	Takes a resource	Releases a resource
When blocked?	If $S \leq 0$, process waits	Never blocked
Also Called	P, down, acquire	V, up, release
Position in code	Before Critical Section	After Critical Section

9.3 Producer-Consumer vs Reader-Writer

Feature	Producer-Consumer	Reader-Writer
Problem	Synchronize buffer access	Allow multiple readers, exclusive writer
Semaphores Used	E, S, I (3 semaphores)	write, read + read_count (2 sema + counter)
Multiple access?	No — only 1 at a time via S	Yes — multiple readers allowed
Overflow/Underflow	Handled by E and I	Not applicable
Real-world example	Buffer/Queue (streaming)	Database reads/writes, notice board

9.4 Busy Waiting vs Blocking

Feature	Busy Waiting	Blocking (Sleep)
Method	while($S \leq 0$) loop	Process sleeps until woken
CPU Usage	High (wastes cycles)	Low (efficient)
Complexity	Simple to implement	More complex
Best For	Very short waits	Long waits
In our code?	Yes (basic semaphore)	Advanced OS kernels

10. Possible Viva Questions

1. What is a semaphore? Who proposed it and when?
2. What is the difference between binary and counting semaphore?
3. What do wait() and signal() operations do?
4. What does atomic operation mean in context of semaphores?
5. What is busy waiting? What is its disadvantage?

6. What is mutual exclusion? How does semaphore achieve it?
 7. What is a critical section?
 8. In Producer-Consumer, what are E, S, and I semaphores?
 9. What happens when E becomes 0 in the Producer-Consumer problem?
 10. What is overflow and underflow in buffer management?
 11. In Reader-Writer problem, can two readers be in CS at the same time?
 12. What does read_count represent in the Reader-Writer problem?
 13. Why does the first reader call wait(write)?
 14. Why does the last reader call signal(write)?
 15. How is the ATM problem similar to the Reader-Writer problem?
-

11. Possible Short Questions

16. Define semaphore with example. (2-3 marks)
 17. What is the difference between P and V operations? (2 marks)
 18. Write the wait() and signal() functions with explanation. (3-4 marks)
 19. What is mutual exclusion? Show with a semaphore example. (4 marks)
 20. Define binary semaphore. How does it work? (3 marks)
 21. What is counting semaphore? Give an example scenario. (3 marks)
 22. What is busy waiting? What is its problem? (2 marks)
 23. What is the critical section problem? (2 marks)
 24. State the rules of the Reader-Writer problem. (3 marks)
 25. What is overflow and underflow? How to prevent them? (3 marks)
-

12. Possible Broad Questions

26. Explain semaphore with its types, operations, and working process. Use examples and code. (10 marks)
 27. Describe the Producer-Consumer problem. Show the solution using semaphores with full code and explanation. (10-12 marks)
 28. Explain the Reader-Writer problem. Provide a solution using semaphores with full code, line-by-line explanation, and rule table. (10-12 marks)
 29. Compare binary semaphore and counting semaphore with examples. Explain wait() and signal() operations with code. (8 marks)
 30. Design a semaphore-based solution for the ATM booth problem. Explain how it maps to the Reader-Writer problem. (8-10 marks)
 31. What is mutual exclusion? How does a semaphore achieve mutual exclusion? Explain with example and diagram. (6 marks)
 32. Explain busy waiting, overflow, underflow and atomic operations with context from synchronization problems. (6 marks)
-

13. Important Code / Pseudocode Summary

13.1 wait() and signal()

```
wait(S) {                               // P operation / down
    while(S <= 0);                       // spin until resource available
    S--;                                 // take the resource
}

signal(S) {                             // V operation / up
    S++;                                 // release the resource
}
```

13.2 Critical Section Template

```
// Before CS
wait(S);

// Critical Section – shared resource access

// After CS
signal(S);
```

13.3 Producer-Consumer Complete

```
// Init: E = N, S = 1, I = 0

// Producer
do {
    wait(E);           // wait if full (overflow check)
    wait(S);           // lock CS
    add();              // CS: add item
    signal(S);          // unlock CS
    signal(I);          // signal: item added
} while (true);

// Consumer
do {
    wait(I);           // wait if empty (underflow check)
    wait(S);           // lock CS
    consume();          // CS: consume item
    signal(S);          // unlock CS
    signal(E);          // signal: slot freed
} while (true);
```

13.4 Reader-Writer Complete

```
// Init: write = 1, read = 1, read_count = 0
```



```
// Writer
wait(write);
writeFunction();
signal(write);

// Reader
wait(read);
read_count++;
if (read_count == 1) wait(write); // first reader blocks writer
signal(read);

readFunction();

wait(read);
read_count--;
if (read_count == 0) signal(write); // last reader unblocks writer
signal(read);
```

13.5 ATM Booth Problem (Reader-Writer pattern)

```
// Init: atm = 1, read = 1, user_count = 0

// Loader (Writer)
wait(atm);
loadMoney();
signal(atm);

// User (Reader)
wait(read);
user_count++;
if (user_count == 1) wait(atm); // first user blocks loader
signal(read);

withdraw();

wait(read);
user_count--;
if (user_count == 0) signal(atm); // last user unblocks loader
signal(read);
```

— End of Notes —